Asynchronous Data Gateway for LoRa-to-Mobile Integration in Disaster Mesh Networks (2026)

Supervisors: Dr. Lillian Wang and Dr. Wai Peng, Wong
Ming Dong, Teh and Jing Hong, Hiew

Abstract: Communication disruption during disasters motivates systems that exchange essential information without cellular service or Internet backhaul. This paper presents a browser-accessible gateway that integrates smartphone location, SOS reports, photographs and two-way text messaging with persistent LoRa delivery. Two Heltec WiFi LoRa 32 V3 nodes provide local HTTPS interfaces and exchange data through their SX1262 radios. The phone browser converts photographs into bounded grayscale JPEGs, while the nodes retain transfer state and acknowledge stored data. We compare baseline stop-and-wait, tuned stop-and-wait, fixed groups with bitmap acknowledgements, and adaptive grouping with acknowledgement recovery. Using the same 2284-byte image, five close-placement trials per mode produced mean receiver-observed completion times of 98.0, 85.8, 77.4 and 65.6 s, respectively. Across one brick wall at approximately 1–2 m, four trials each produced means of 105.25 s for fixed grouping and 88.00 s for adaptive grouping; all eight transfers completed. Adaptive grouping reduced mean time by 15.2% and 16.4% relative to fixed grouping in the two placements, although its timing variability was greater. These descriptive results support further investigation of the integrated policy, while limited sample sizes, manual timing and the absence of packet traces constrain causal and reliability claims. Browser compatibility and certificate handling remain deployment considerations.

Index Terms: Adaptive transmission, disaster communication, image compression, LoRa, mobile gateway, selective retransmission.

I. Introduction and Literature Review

A. Background and Problem Definition

Emergency communication must accommodate users who cannot depend on cellular coverage, Internet backhaul or a previously installed specialist application. Smartphone networking for disaster recovery has been investigated in TeamPhone [1], while LOCATE combines mobile interaction with LoRa communication for emergency management [2]. These studies establish the relevance of mobile access and infrastructure-independent communication. They also show that disaster messaging and multihop forwarding are established research topics rather than sufficient novelty claims for a new prototype.

The engineering problem addressed here is the integration of accessible phone interaction with reliable transfer of small, useful records over a constrained radio link. A short SOS can describe an emergency and its location, while a photograph may provide additional context. Image delivery, however, requires fragmentation and repeated feedback exchanges. Reducing the transmitted image size and reducing acknowledgement overhead are distinct interventions; their effects must be evaluated separately. The system must also distinguish a report saved locally from one stored by the destination.

Our implementation uses an HTTPS page served by each LoRa node. The phone performs image preparation and requests a location estimate through the browser. The embedded node stores reports and schedules subsequent transmissions. This allocation allows accepted transfers to continue while the interface awaits progress, although the firmware still uses cooperative servicing and can experience blocking web operations. The term asynchronous describes this separation of submission, transmission and acknowledgement; it does not imply that every operation is nonblocking.

B. Existing Architectures and Positioning Approaches

The IEEE Wireless Resilience study integrates a LoRa sensor network, GPS-capable emergency hardware and a mobile rescue application [3]. LoRaAid instead investigates joint communication and localisation using LoRa measurements [4]. These architectures address different positioning requirements. A receiver attached to a node can locate that node independently of a phone, whereas a browser can request the position of the person using the phone. Neither approach establishes the accuracy or availability of the other.

Chan Xin Yen et al. proposed phone-derived location and a LoRa-connected web form, but documented difficulties obtaining browser location because of HTTPS and certificate restrictions [5]. This directly related non-IEEE study is retained because it identifies a deployment issue encountered by the present design. Our successful local HTTPS interaction addresses that implementation question in the tested environment; it does not establish that phone location over LoRa is a new concept.

Existing platforms also constrain claims about access and forwarding. Meshtastic is an open-source system with browser access through HTTP, Bluetooth or serial connections [6]. Its ordinary CLIENT role can rebroadcast packets, enabling forwarding through participating users' nodes [7]. LoRaWAN uses gateways and network-server functions, and supports private deployment models [8]. Consequently, neither civilian relaying nor browser access alone differentiates our work. The present study examines a particular SOS workflow and a reproducible comparison of transfer configurations on the same hardware.

C. Image Transmission and the Research Gap

Chen et al. investigated batching, bit-vector acknowledgements and selective retransmission for LoRa image transfer [9]. Torres et al. also examined a multi-packet LoRa image protocol [10]. These studies motivate fixed grouping as the principal transport baseline: comparing an adaptive method only against stop-and-wait would conflate the benefit of grouping with the benefit of adaptation. Dede et al. studied JPEG-based image transmission and reconstruction over LoRa [11], reinforcing the need to keep compressed payloads constant during a protocol comparison.

More recent IEEE work explores different interventions. Onykiienko et al. combine Haar-wavelet preprocessing with JPEG on ESP32-CAM hardware [12], while De et al. investigate interleaved reception using two frequencies and receivers [13]. These approaches motivate alternative ways to reduce delay, but their hardware and preprocessing differ from our single-transceiver, phone-assisted architecture. Their published performance is therefore not a matched benchmark for our measurements. Table I summarises the relevant distinctions.

TABLE I
RELATED WORK AND ITS RELEVANCE

Approach	Established capability	Implication for this study
LOCATE and TeamPhone [1], [2]	Mobile disaster communication	Mobile access and relaying have precedent.
Chen and Torres [9], [10]	Multi-packet image transfer	Compare adaptation against fixed grouping.
Onykiienko [12]	Wavelet and JPEG processing	Separate compression gains from transport gains.
De [13]	Interleaved reception	Additional receivers change the hardware comparison.
Chan et al. [5]	Phone location and web access	Evaluate HTTPS access and certificate handling.

The research question is whether an integrated browser gateway with adaptive group selection and acknowledgement recovery reduces receiver-observed image-reception time relative to fixed grouping, while preserving stored and verified delivery. We hypothesise a lower mean duration under the evaluated conditions, without assuming that adaptation always outperforms a suitable fixed group. The objectives are to implement the complete SOS workflow, document its processing and delivery semantics, compare four configurations with identical compressed bytes, and assess the scope of the resulting evidence. The candidate contribution is this integration and its measured behaviour, rather than a new image codec or the invention of bitmap acknowledgements.

II. METHODOLOGY

A. Prototype and System Architecture

We used two Heltec WiFi LoRa 32 V3 boards, each containing an ESP32-S3 and an SX1262 transceiver. One board served the civilian interface and the other the rescuer interface. Each radio used one antenna for alternating transmission and reception. Phones joined an open local Wi-Fi access point, and the node supplied the web interface and application endpoints over HTTPS. Figure 1 identifies the two communication links and the persistent source and destination copies.

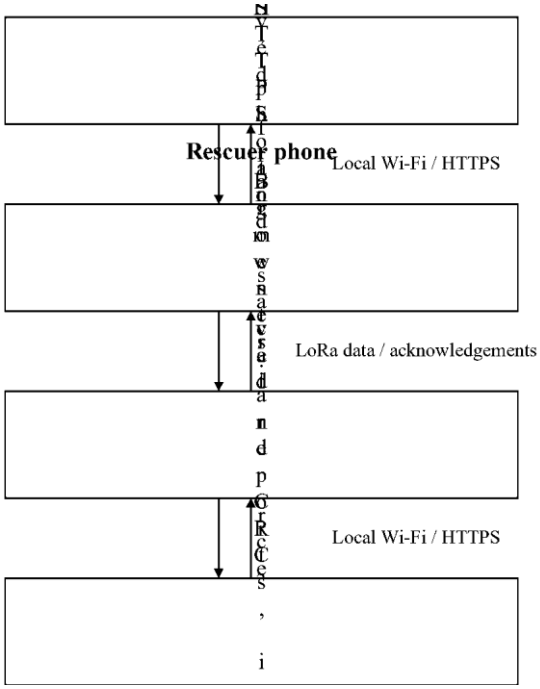


Fig. 1. System architecture. Phones access their respective nodes locally; the nodes exchange reports, image fragments and acknowledgements over LoRa. The measured path contains two nodes and no intermediate relay.

The implementation includes forwarding through civilian nodes, bounded hop counts and duplicate suppression. However, the quantitative experiments used a direct two-node path. Mesh capability in the software is therefore distinguished from experimentally demonstrated multihop performance. All participating nodes require compatible protocol, radio and authentication settings. Table II records the inspected source configuration; per-run firmware logs were not retained to independently verify the exact flashed state.

TABLE II
HARDWARE AND SOURCE CONFIGURATION

Parameter	Value	Reason for documenting it
Controller and radio	ESP32-S3; SX1262	Defines memory and radio resources.
Radio configuration	915 MHz; 125 kHz; SF9; coding rate 4/7	Determines nominal frame duration.
Output and preamble	22 dBm; 8 symbols	Affects link budget and airtime.
Image payload	180 bytes per full chunk	Held constant across modes.
Accepted JPEG	At most 8192 bytes	Bounds radio and storage workload.
Adaptive airtime budget	8000 ms per data group	Limits time before group feedback.

Parameter	Value	Reason for documenting it
Application-task stack	16 KiB	Accommodates combined service paths.

These are prototype settings, not evidence of regulatory certification or region-wide operating permission. We do not carry forward the earlier proposal's AS923 duty-cycle-compliance claim because the measured study did not validate such a scheduler.

B. Software Responsibilities and Delivery Semantics

Figure 2 shows the implemented software responsibilities. Browser JavaScript manages form input, image preparation, location requests and interface polling. WebServerManager validates requests and invokes StorageManager and LoRaMeshManager. StorageManager maintains records, images and progress in LittleFS. LoRaMeshManager manages queues and acknowledgements, TransferPolicy selects grouped-transfer behaviour, and RadioLib controls the transceiver. DNS and HTTP redirection support portal discovery; captive-portal behaviour remains dependent on the phone.

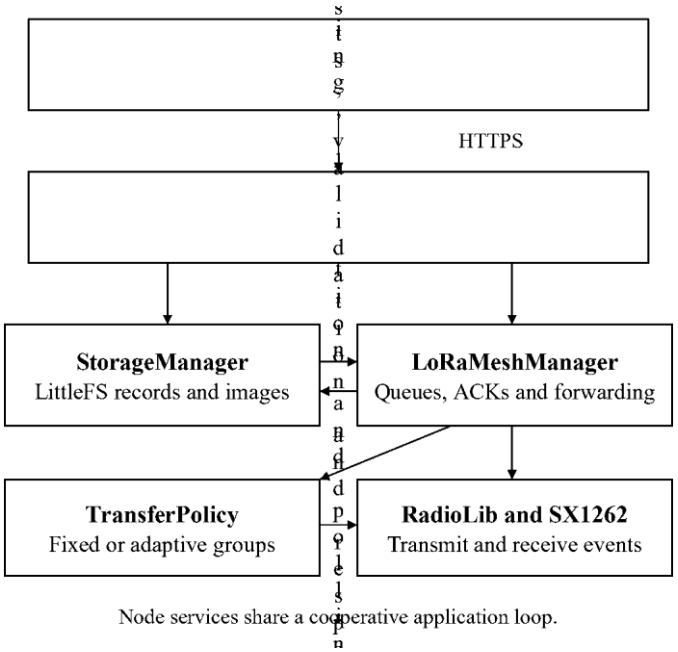


Fig. 2. Software architecture. The browser executes on the phone; web handling, persistent storage and radio scheduling execute on the node. Radio events are processed through the shared application loop.

An SOS is saved before its optional photo is uploaded. This ordering prevents image preparation from delaying local acceptance of the shorter emergency report. Photo transmission begins after the associated SOS has an acknowledged rescuer destination. A receiver writes and reads back each chunk before saving its reception bit. Final completion requires the expected byte count, JPEG boundary-marker checks, CRC32 verification and persisted completion state. The sender marks destination delivery after the corresponding acknowledgement. These checks establish consistency of the selected JPEG bytes, rather than the usefulness of the photographed scene.

Persistent progress supports retry after interrupted exchanges. Duplicate logical messages are recognised rather than displayed repeatedly. An acknowledgement confirms stored progress or destination completion; it does not confirm that a rescuer has read the report or performed a rescue. SOS and chat retain scheduling priority over images. This is an implemented scheduling rule, whose latency under concurrent traffic has not yet been measured.

C. Browser Image Compression

Image processing runs in the browser using the phone's processor and memory. Figure 3 separates the application-controlled search from the JPEG encoder supplied by the browser. The input is selected from a camera or existing file. Encoded files exceeding 15 MiB are rejected before image decoding. A decoded photograph can occupy substantially more memory than its compressed file, so this input limit does not establish a peak-memory bound.

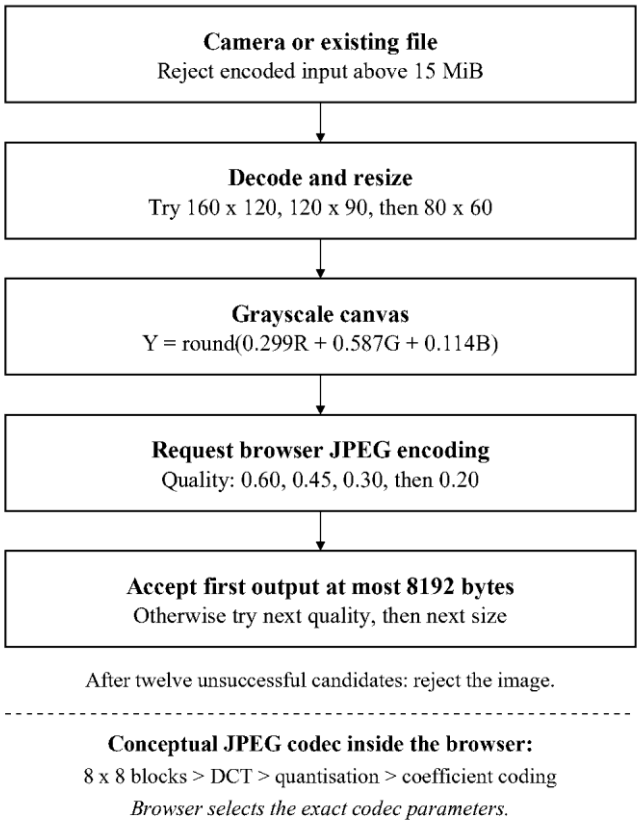


Fig. 3. Image-compression architecture. The application tries size and quality candidates in order and accepts the first JPEG of at most 8192 bytes. The lower path shows the conceptual JPEG stages; codec internals are selected by the browser.

For each bounding rectangle, the scale is

$$s = \min\left(\frac{W_b}{W}, \frac{H_b}{H}, 1\right) \quad (1)$$

where W and H are the original dimensions and W_b and H_b are the current bounds. The minimum preserves aspect ratio and prevents enlargement. The application tries 160 × 120, 120 × 90 and 80 × 60 pixel bounds in that order,

rounding output dimensions and retaining at least one pixel on each axis. These choices prioritise a compact preview and bound the candidate search; they are design parameters rather than experimentally optimised values.

The resized image is drawn onto a white canvas and converted using

$$Y = \text{round}(0.299R + 0.587G + 0.114B) \quad (2)$$

The coefficients follow a conventional brightness-weighted conversion discussed in recent IEEE/CVF work [14]. The same Y value is written into all three colour channels. This removes colour variation before encoding, but does not guarantee that the browser produces a single-component JPEG. It also removes potentially useful colour cues, which is a substantive application trade-off.

At each size, canvas.toBlob requests JPEG quality settings 0.60, 0.45, 0.30 and 0.20, accepting the first output no larger than 8192 bytes. If all four fail, the next smaller rectangle is tried; failure of all twelve candidates rejects the photo. The quality argument influences the browser encoder but is neither a compression ratio nor a portable measure of visual quality [15]. The search retains the first acceptable candidate rather than proving a globally optimal image. The 8192-byte bound limits transmission and storage demand across the four modes.

The standard baseline JPEG model explains why this process reduces bytes. Samples are organised into 8 × 8 blocks and transformed into spatial-frequency coefficients using the discrete cosine transform [16], [17]. The transform concentrates common smooth-image structure into relatively few coefficients. Quantisation then applies

$$Q(u, v) = \text{round}\left(\frac{F(u,v)}{S(u,v)}\right) \quad (3)$$

where F is a transform coefficient and S its quantisation step. Larger steps discard more coefficient precision and often create more zeros; this is a principal source of irreversible JPEG loss. Zigzag ordering groups low- and high-frequency coefficients for subsequent representation, while differential DC coding, zero-run representation and Huffman coding reduce coding redundancy in the baseline model [17]. Entropy coding preserves the already quantised values; it does not restore detail removed by resizing or quantisation.

Our application delegates those codec stages to the browser and does not select its exact tables, sampling factors or entropy-coding configuration. Consequently, equal source photographs or quality settings need not produce identical bytes across browsers. Transport experiments must reuse the same prepared JPEG. We did not measure PSNR, structural similarity or rescue-task recognition accuracy; image fidelity remains distinct from CRC-verified transfer integrity.

D. Phone-Derived Location and HTTPS

The page checks whether the browser recognises a secure context and exposes navigator.geolocation before requesting coordinates. It calls getCurrentPosition with high accuracy requested, a 12 s timeout and a maximum cached-position age of 30 s. W3C specifies permission and secure-context requirements, while leaving the positioning provider to the implementation [18], [19]. High accuracy is a request to the provider, not proof of a fresh satellite fix. Allowing a recent cached estimate reduces repeated acquisition demand, but limits claims about freshness.

Latitude and longitude are validated as a pair within geographic ranges and included in the SOS. Manual coordinates and written location remain available when automatic acquisition fails. We observed automatic coordinate population with mobile data disabled after continuing past the node's certificate warning; no certificate or dedicated application was installed. Chrome was used on Xiaomi 12T and Xiaomi 17T phones during the project. Browser and operating-system versions were not recorded, and the offline-location observation was not separately logged for each device.

HTTPS supplies the local transport and secure-context prerequisite; the phone's location provider supplies the estimate. We did not retain its provider, accuracy radius or acquisition timestamp. Thus, this observation establishes successful browser interaction under the tested conditions, without establishing positioning accuracy, GPS-only operation or universal acceptance of self-signed certificates. It does not imply that HTTPS requires Internet access.

E. Four Transfer Configurations

All modes retain the same image bytes, fragmentation, stored-progress requirements and completion verification. A 2284-byte image requires ceiling(2284/180) = 13 data chunks: twelve full chunks and one carrying 124 bytes. The ceiling is necessary because a nonempty remainder requires another chunk. This count excludes acknowledgements, manifests and retransmissions. Table III and Fig. 4 distinguish the four scheduling policies.

TABLE III
TRANSFER CONFIGURATIONS

Mode	Feedback and group rule	Principal difference
1	One chunk, then acknowledgement	Baseline scheduling and retry timing.
2	One chunk, then acknowledgement	Shorter delays and image timer after transmission.
3	Up to four chunks, then bitmap	Fixed grouping and selective retry.
4	Adaptive group, then bitmap	Rate-based selection and acknowledgement recovery.

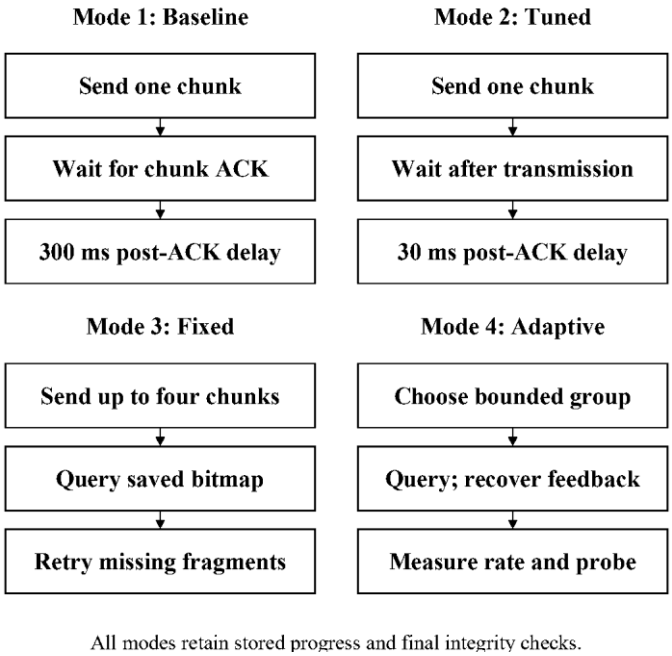


Fig. 4. Logical transfer paths for the four modes. All paths share manifest exchange, persistent reception and final integrity verification. Retry arrows are abbreviated.

Mode 1 retains a 200 ms planning interval and 300 ms post-image-acknowledgement delay. Mode 2 reduces these to 20 ms and 30 ms and starts its image acknowledgement timer after local transmission. This comparison tests scheduling overhead without changing the chunk size. Mode 3 sends groups of at most four missing chunks and requests a bitmap that identifies stored fragments. Missing fragments can then be selectively retransmitted rather than repeating the complete image. A final group may contain fewer than four chunks.

F. Adaptive Group Selection and Recovery

Mode 4 begins each image with a preferred group of two chunks. Its measured rate is

$$r_g = \frac{1000 B_{new}}{\Delta t_g} \quad (4)$$

where B_new is newly confirmed image data in bytes and delta_t_g is the elapsed group interval in milliseconds, including waits and retries. Counting only new bytes prevents retransmissions from inflating the useful-delivery estimate; multiplying by 1000 gives bytes per second. This internal feedback quantity is separate from the manually measured interface duration.

After its first eligible observation, the policy probes another group size. Later probes normally occur after three measured groups at the preferred size. Candidate sizes are obtained by doubling or halving the preference, within the permitted bounds. A candidate is retained only if its measured rate improves by more than 5%. Preferred-rate smoothing assigns weights of 0.75 to the previous estimate and 0.25 to the new measurement. The threshold reduces switching on small observed differences, while smoothing limits dependence on a single group. These are heuristic design choices requiring sensitivity analysis, not proven optimal constants.

The maximum group is bounded by

$$w_{max} = \min(8, \max(1, \lfloor \frac{8000}{\tau_c} \rfloor)) \quad (5)$$

where tau_c is the calculated duration of a full chunk frame in milliseconds. The floor selects a whole number of frames within the data-airtime budget, and the lower bound permits at least one frame. With the inspected configuration, the estimated 1574 ms frame duration gives a cap of five chunks. Listening intervals and control frames add elapsed time, and a frame longer than the budget would itself exceed it. The budget is therefore a scheduling bound rather than a regulatory duty-cycle guarantee.

Figure 5 summarises recovery and adaptation. One unanswered bitmap query is repeated before the preferred group is reduced. Two consecutive unanswered queries allow a smaller preference for subsequent groups. Further silence invokes the existing retry pause and manifest recovery. The acknowledgement timer uses smoothed reply delay and variation, a path-dependent floor and a 60 s ceiling. Missing feedback does not reveal whether data or the reply was lost; retrying the query can recover the destination's saved bitmap without immediately repeating all data.

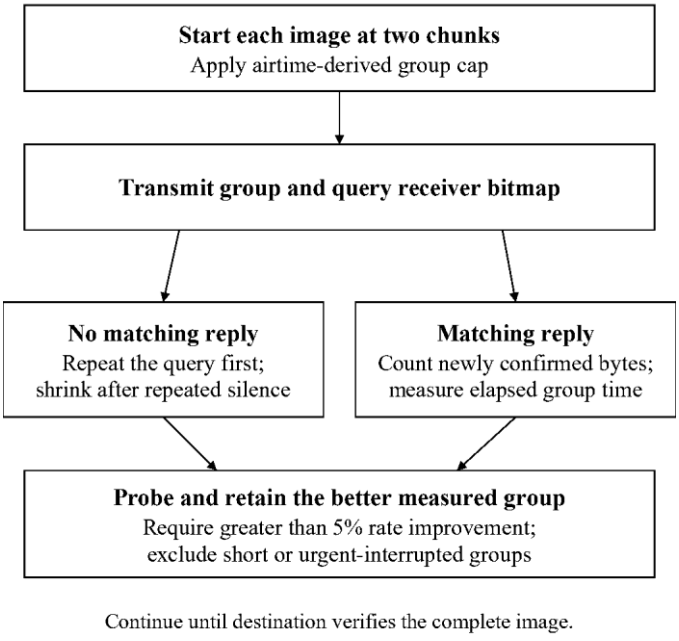


Fig. 5. Adaptive policy. The sender measures newly confirmed bytes, probes bounded group sizes and recovers missing bitmap replies. Short final groups and groups interrupted by urgent traffic do not train the rate estimate.

The policy resets per image. Short final groups, manifest responses and groups interrupted by urgent traffic are excluded from training because their size or duration is constrained by factors other than the candidate group size. A path-state change or receiver rollback resets the estimate. The path estimate uses acknowledgement hop information for timing; it is not an additional route-selection algorithm. On the direct experimental link, all modes used the same two endpoints.

G. Experimental Procedure and Analysis

We first recorded five trials per mode with the boards side by side, giving 20 timings. We then recorded four trials each for modes 3 and 4 in adjacent rooms, approximately 1–2 m apart, with one intervening brick wall. Both conditions used the same 2284-byte JPEG. All eight obstruction attempts completed. Earlier timings obtained with different compressed images were excluded from the comparative dataset because payload differences would confound protocol effects.

The common payload fixes scene content, encoded size and nominal chunk count. Repetition avoids dependence on one observation; Chen et al. likewise used five repeated measurements in their testbed study [9]. This precedent does not establish adequate statistical power for our experiment. Four obstruction runs per mode are the measurements completed so far. We preserve every genuine reading, including the 113 s baseline result, and make no paired-trial assumption because acquisition order was not recorded.

The measured interval is

$$T = t_{\text{complete and verified}} - t_{\text{reception indication}} \quad (6)$$

Both endpoints were observed on the rescuer interface and timed manually in seconds. This definition measures the wait for an incoming image to become available with verified completion and avoids synchronising node clocks. The approximately 3 s initial receiving indication observed during the close-placement experiment precedes this timer. The interval excludes phone image preparation and does not end at the civilian's final acknowledgement. Interface polling and manual reaction time can affect the observations.

For each mode and condition, we calculated

$$\overline{T} = \frac{\sum T_i}{n}$$

$$s_T = \sqrt{\frac{\sum (T_i - \overline{T})^2}{n - 1}} \quad (7)$$

The arithmetic mean weights each run equally and retains the contribution of slow completions. Sample standard deviation describes variation around that mean in seconds. The n−1 denominator accounts for estimating the mean from the same sample: it equals four for the close-placement condition and three for obstruction. Under independent sampling with finite variance, this corrects bias in the sample variance; it does not establish independence or provide a confidence interval. We use the statistics descriptively.

The reduction relative to a reference-mode mean is

$$R = 100 \frac{T_{\text{reference}} - T_{\text{comparison}}}{T_{\text{reference}}} \quad (8)$$

Using the reference mean as denominator makes the direction of comparison explicit. We also report seconds saved to retain practical scale. Percentages describe elapsed-time reduction rather than increased radio bit rate. Completion is reported as completed attempts divided by attempted transfers when both counts are known. This prevents successful durations from concealing failed attempts. Packet-delivery ratio, retransmission counts, measured airtime and energy were not recorded in these trials.

III. Results

A. Close-Placement Comparison

Table IV preserves the individual readings and their summaries. Adaptive grouping had the lowest mean, followed by fixed grouping, tuned stop-and-wait and baseline stop-and-wait. Relative to mode 1, modes 2, 3 and 4 reduced mean duration by 12.4%, 21.0% and 33.1%, respectively. The principal mode-4 versus mode-3 comparison gave an 11.8 s reduction, equivalent to 15.2% of the fixed-group mean.

TABLE IV
Close-Placement Timings for the Same 2284-Byte Image

Mode	Individual times (s)	Mean ± sample SD (s)
1	94, 90, 95, 98, 113	98.0 ± 8.86
2	85, 89, 84, 83, 88	85.8 ± 2.59
3	75, 80, 77, 82, 73	77.4 ± 3.65
4	58, 71, 75, 63, 61	65.6 ± 7.13

Mode 4 varied more than mode 3, with sample standard deviations of 7.13 and 3.65 s. Mode 2 had the smallest standard deviation, 2.59 s. Thus, the lowest average duration and the most consistent timings occurred in different configurations. The earlier close-placement record does not establish a complete failure denominator, so a success percentage is not inferred for this condition.

B. Brick-Wall Obstruction

Table V presents the separate obstruction dataset. Mode 4 averaged 88.00 s compared with 105.25 s for mode 3, a reduction of 17.25 s or 16.4%. Both modes completed four of four attempts, with zero failed completions. Their observed completion fractions were therefore both 100% within this session. Successful completion may still include retransmitted packets and delayed acknowledgements.

TABLE V
Brick-Wall Timings at Approximately 1–2 m

Mode	Individual times (s)	Mean ± sample SD (s)
3	101, 116, 98, 106	105.25 ± 7.89
4	76, 101, 85, 90	88.00 ± 10.42

Mode 4 again showed greater timing variation. Both obstruction means exceeded their close-placement counterparts, but separation, room geometry and session also changed. Without an unobstructed control at the same placement, the difference cannot be attributed solely to the wall. Figure 6 displays all observations to make both the mean ordering and variation visible.

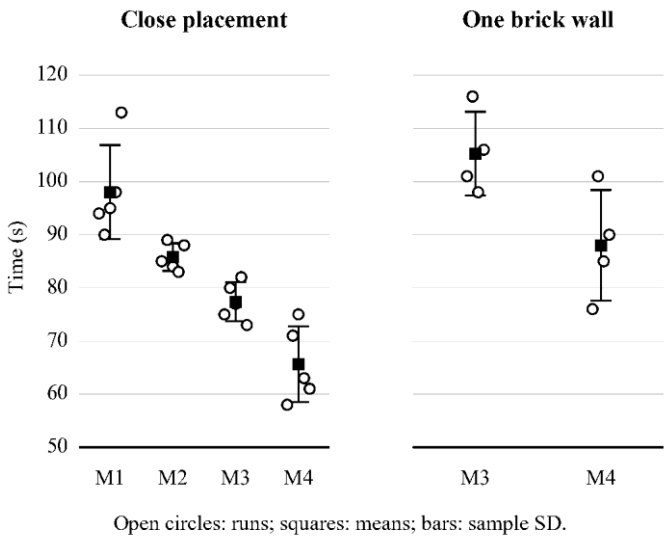


Fig. 6. Individual receiver-observed times, with mean and sample-standard-deviation markers. The two physical conditions are separate datasets; error bars describe sample variation rather than confidence intervals.

C. Distance and Location Observations

We checked all four modes over approximately 50–100 m with a clear path and observed broadly similar delivery behaviour. Per-distance numerical timings and attempt counts were not retained, so this remains a qualitative observation. It establishes neither a maximum range nor a distance-performance curve. The successful phone-coordinate interaction described in Section II-D is likewise a functional observation, separate from quantitative positioning accuracy and browser-wide compatibility.

IV. Discussion

A. Why Use a Local Browser Gateway

The design prioritises access by a person carrying an ordinary phone without a dedicated application, additional positioning receiver or USB cable. A local web page centralises the interface on the node and permits the same application code to serve different browser platforms. Moving image preparation to the phone avoids decoding a full camera photograph on the ESP32. After upload, the node owns the saved record and transfer state. These are architectural benefits; the study did not measure onboarding time, total energy or deployment cost.

A native Android application with USB OTG remains a reasonable alternative for managed responder equipment. It can expose hardware access and a controlled application lifecycle, but requires application distribution, a physical connection, USB-host support and device permission handling [20]. Native BLE access avoids the cable but still requires application provisioning and device association. Browser access over Web Bluetooth or Web Serial has more restricted platform support than ordinary web interaction [21], [22]. Our Wi-Fi/HTTPS choice avoids those two browser APIs, while introducing access-point power consumption, certificate handling and web-service load. The experiment does not establish that Wi-Fi is more reliable or more energy-efficient than BLE or USB.

A private LoRaWAN installation is appropriate where gateway and network-server services can be deployed and maintained [8]. The two-node prototype instead aims to carry its local access and transfer functions on the participating boards. This reduces additional service dependencies for the examined topology, but gives up the interoperability and operational support of an established network stack. Meshtastic already provides open-source messaging, forwarding and web access [6], [7]. We implement a separate protocol to control the four image-transfer policies and SOS completion semantics for this study, not because existing systems lack civilian relaying or are inherently unsuitable for emergency communication.

Phone-derived location is appropriate when a user's position is needed and a capable phone is already present. Node-mounted GNSS remains preferable when the node must locate itself without a phone. Radio-derived localisation addresses different environments and infrastructure assumptions, as illustrated by LoRaAid [4]. Our browser approach avoids adding a GNSS module to each node, but its availability depends on phone permissions, positioning services and browser security decisions. No component-cost or energy advantage has been quantified.

B. Why Adapt Groups Rather Than Only Compress More

Further compression reduces bytes at the cost of visual information. The present grayscale preview already discards colour and spatial detail; reducing it again could obscure features relevant to an SOS. Wavelet preprocessing and alternative encoders warrant separate fidelity and processing-cost experiments [12]. Interleaved reception can address waiting through additional receiver resources [13]. Our policy instead changes feedback scheduling while keeping compressed bytes, chunk size and transceiver count fixed, allowing the observed configurations to be compared on the same payload.

Fixed grouping amortises acknowledgement overhead, as established in [9], [10], and may be preferable when one group size performs well consistently. Adaptive grouping seeks to respond to observed delivery rate and feedback delay. Its probing, conservative initial group and recovery can themselves add delay, especially for a 13-chunk image with few learning opportunities. The observed lower means therefore support the implemented configuration in these placements, rather than a claim that adaptation must always be faster.

The current comparison also changes more than one mechanism. Mode 4 combines group selection, reply-delay estimation and recovery, and its effective cap is five chunks compared with mode 3's four. Some benefit may arise from the larger allowed group. Fixed groups of two, four and five, an adaptive policy capped at four, and separate recovery-only and selection-only variants are needed to isolate these effects. The candidate contribution is the combined policy's evaluated

behaviour; the present data cannot assign its benefit to one component.

C. Browser Compatibility and Practical Access

The application cannot presently be described as working on all browsers. We used Chrome on Xiaomi 12T and Xiaomi 17T phones, but did not perform a version-controlled compatibility campaign. Table VI distinguishes those observations from candidate platforms. MDN compatibility data document Geolocation, Canvas.toBlob and image.decode support in current major browser families [23]. That API evidence does not verify this complete application or its self-signed-certificate workflow on every platform.

TABLE VI
BROWSER SCOPE AND REMAINING VALIDATION

Browser and platform	Project evidence	Qualification
Chrome on Android	Used on Xiaomi 12T and 17T	Versions unrecorded; no exhaustive feature matrix.
Firefox on Android	Not tested	Required API support documented; validate node access.
Samsung Internet	Not tested	Required API support documented; validate node access.
Edge on Android	Not tested	Validate the installed mobile build and certificate path.
Opera on Android	Not tested	Required API support documented; validate node access.
Safari on iPhone/iPad	Not tested	API support exists; validate HTTPS and offline location.
Chrome/Edge/Firefox on iOS	Not tested	Validate each app and OS combination.
Desktop Chrome/Edge/Firefox/Safari	Not tested	File upload is a candidate; camera and location differ.
Captive-portal mini-browser	Not a validated target	Permissions and storage require separate checks.

Compatibility has several independent requirements. First, the phone must remain connected to the node's local Wi-Fi network. Second, the browser must accept the HTTPS origin and recognise a secure context; continuing past a warning is not a portable certificate-provisioning strategy. Third, geolocation requires permission and an available positioning provider [18], [19]. Finally, the image must decode within device memory, and the page must support the JavaScript and canvas operations used by the application. Supporting Geolocation alone is insufficient to establish full compatibility.

The camera control uses a file input with capture="environment", which requests a suitable camera rather than guaranteeing a particular camera interface. MDN documents differing behaviour across devices, with desktop browsers commonly presenting a file picker [24]. The separate gallery option remains useful when capture is unavailable or memory pressure prevents a new photo. If location permission is denied, manual coordinates or written location preserve the reporting workflow. Persistent browser storage retains the report-access token; clearing site data can remove that access even while the node retains the report.

D. Interpretation, Limitations and Future Evaluation

The results answer the latency question narrowly: mode 4 produced lower mean receiver-observed times than mode 3 in both measured placements, with greater variation. Equal successful-completion counts in the obstruction session do not distinguish reliability, and neither speed nor timing consistency measures packet loss directly. We also cannot infer energy savings from shorter duration without measuring device activity and power.

Manual timing, interface polling, unknown trial order, missing payload hashes and unrecorded per-run firmware state limit reproducibility. Antenna orientation, wall thickness, pending traffic and web-client workload were not fully documented. Development logs showed web-service intervals of several seconds, so cooperative web servicing is a plausible source of radio-processing delay. Those cumulative diagnostics are not synchronised with the reported trials and cannot explain a particular slow completion. Earlier stack exhaustion and HTTPS failures motivated a larger application stack and connection-handling repairs; they should not be concealed by reporting only later successful trials.

The study concerns one small image, two boards and limited placements. Four or five observations per configuration provide descriptive evidence, without establishing statistical significance, dense-network scalability, multihop performance or field-rescue effectiveness. A future evaluation should interleave mode order, retain the exact JPEG and firmware identifiers, define a stopping rule, collect packet and acknowledgement traces, and use matched physical controls. Multiple image sizes would examine adaptation overhead as the number of groups increases. A browser campaign should record device and software versions, certificate treatment, permissions, camera and file paths, offline location, reconnection and memory failures. These measurements would test the conditions under which the present benefits persist or disappear.

V. CONCLUSION

We implemented a local HTTPS gateway integrating phone-derived location, browser image compression, SOS reporting and two-way messages with persistent LoRa delivery. The architecture separates phone preparation from node storage and scheduled transmission, while exposing destination-confirmed progress. With the same 2284-byte JPEG, the adaptive configuration reduced mean receiver-observed image time by 15.2% relative to fixed grouping in close placement and by 16.4% across one brick wall. All eight obstruction attempts completed, although adaptive timings varied more than fixed-group timings. The evidence supports a preliminary latency benefit for the complete configuration

rather than universal superiority or improved packet reliability. Browser access was used through Chrome on two Xiaomi phones; wider compatibility remains to be validated. The next research priority is to isolate group-size and recovery effects through matched experiments and to extend evaluation across payload sizes, browser platforms and multihop paths.

ACKNOWLEDGMENT

We thank Dr. Lillian Wang and Dr. Wai Peng, Wong for their supervision. OpenAI Codex assisted with software development, literature discovery, manuscript drafting, diagrams and statistical summaries. We conducted the hardware experiments and recorded the measurements.

REFERENCES

[1] Z. Lu, G. Cao, and T. La Porta, "TeamPhone: Networking SmartPhones for Disaster Recovery," *IEEE Transactions on Mobile Computing*, vol. 16, no. 12, pp. 3554–3567, Dec. 2017, doi: 10.1109/TMC.2017.2695452.

[2] L. Sciullo, F. Fossemò, A. Trotta, and M. Di Felice, "LOCATE: A LoRa-based mObile emergenCy mAnagement sysTEm," in *2018 IEEE Global Communications Conference (GLOBECOM)*, Abu Dhabi, United Arab Emirates, 2018, pp. 1–7, doi: 10.1109/GLOCOM.2018.8647177.

[3] M. S. Imran, M. Daniel, Z. S. Attarbashi, D. O. D. Handayani, S. Iftikhar, and D. Al-Madani, "Wireless Resilience: A LoRa-Based Sensor Network and Mobile Application for Flood Victim Rescue System," in *2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED)*, Kuala Lumpur, Malaysia, 2023, pp. 1–6, doi: 10.1109/ICCED60214.2023.10425043.

[4] H. Zhang, G. Liu, Y. Xu, and T. Jiang, "LoRaAid: Underground Joint Communication and Localization System Based on LoRa Technology," *IEEE Transactions on Wireless Communications*, vol. 23, no. 5, pp. 5248–5260, May 2024, doi: 10.1109/TWC.2023.3325330.

[5] Chan Xin Yen, K. Shanmugam, and M. E. Rana, "Short-Term Mobile Ad Hoc Network with LoRa Based Infrastructure in Disaster Relief," *Journal of Applied Technology and Innovation*, vol. 6, no. 2, pp. 55–59, 2022, doi: 10.65136/jati.v6i1.170. Publisher full text.

[6] Meshtastic Project, "Web Client Overview." [Online]. Available: https://meshtastic.org/docs/software/web-client/. Accessed: Oct. 19, 2026.

[7] Meshtastic Project, "Device Configuration." [Online]. Available: https://meshtastic.org/docs/configuration/radio/device/. Accessed: Oct. 19, 2026.

[8] LoRa Alliance, "About the LoRaWAN Standards." [Online]. Available: https://lora-alliance.org/lorawan-for-developers/. Accessed: Oct. 19, 2026.

[9] T. Chen, D. Eager, and D. Makaroff, "Efficient Image Transmission Using LoRa Technology In Agricultural Monitoring IoT Systems," in *2019 International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, Atlanta, GA, USA, 2019, pp. 937–944, doi: 10.1109/iThings/GreenCom/CPSCom/SmartData.2019.00166.

[10] J. Torres, M. Nunez, and J. M. Mauricio Villanueva, "LoRa Multi-Packet Protocol for Image Transmission," in *2024 8th International Symposium on Instrumentation Systems, Circuits and Transducers (INSCIT)*, Joao Pessoa, Brazil, 2024, pp. 1–6, doi: 10.1109/INSCIT62583.2024.10693388.

[11] O. L. Dede, H. M. Jalajamony, and R. E. Fernandez, "Image Transmission over LoRa-Based Networks: A Performance Study Using Image Compression and Reconstruction Methods," in *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, 2024, pp. 642–643, doi: 10.1109/CCNC51664.2024.10454687.

[12] Y. Onykiienko, S. Naida, M. Mazin, I. Kyrylenko, and P. Popovych, "An Efficient Image Compression Method for LoRa Networks," in *2026 IEEE 8th International Conference on Smart Technologies in Power Engineering and Electronics (STEE)*, Kyiv, Ukraine, 2026, pp. TT2.14.1–TT2.14.5, doi: 10.1109/STEE63704.2026.11663995.

[13] S. De *et al*., "Interleaved Reception in LoRa Using IFT: A Hardware-Efficient Architecture for Fast and Low-Power Image Transmission," *IEEE Internet of Things Journal*, vol. 12, no. 19, pp. 39777–39787, Oct. 2025, doi: 10.1109/JIOT.2025.3588064.

[14] L. Stracke, L. Nimmermann, S. Agnihotri, M. Keuper, and V. Blanz, "Vision At Night: Exploring Biologically Inspired Preprocessing For Improved Robustness Via Color And Contrast Transformations," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, 2025. Author manuscript, version 1.

[15] MDN Web Docs, "HTMLCanvasElement: toBlob() method." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/HTMLCanvasElement/toBlob. Accessed: Oct. 19, 2026.

[16] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete Cosine Transform," *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, Jan. 1974, doi: 10.1109/T-C.1974.223784.

[17] G. K. Wallace, "The JPEG Still Picture Compression Standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992, doi: 10.1109/30.125072.

[18] W3C, "Geolocation," Candidate Recommendation Snapshot, Mar. 26, 2026. [Online]. Available: dated specification. Accessed: Oct. 19, 2026.

[19] W3C, "Secure Contexts," Candidate Recommendation Draft, Nov. 10, 2023. [Online]. Available: dated specification. Accessed: Oct. 19, 2026.

[20] Android Developers, "USB host overview." [Online]. Available: https://developer.android.com/develop/connectivity/usb/host. Accessed: Oct. 19, 2026.

[21] MDN Web Docs, "Web Bluetooth API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Bluetooth_API. Accessed: Oct. 19, 2026.

[22] MDN Web Docs, "Web Serial API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API. Accessed: Oct. 19, 2026.

[23] MDN Web Docs contributors, "Browser compatibility data: Geolocation, HTMLCanvasElement and HTMLImageElement." [Online]. Available: https://github.com/mdn/browser-compat-data. Accessed: Oct. 19, 2026.

[24] MDN Web Docs, "capture HTML attribute." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/capture. Accessed: Oct. 19, 2026.